Server Side Development using Node.js and Express.js

Prepared By
Asst. Prof. Belim Irfan

Unit 1 Introduction to Node.js

Web Applications

A web application, as you may already know, is a program that runs on a server and is rendered by a client browser, using the internet to access all the resources of that application. It usually can be easily broken down into three parts:

1. Client
2. Server
3. Database

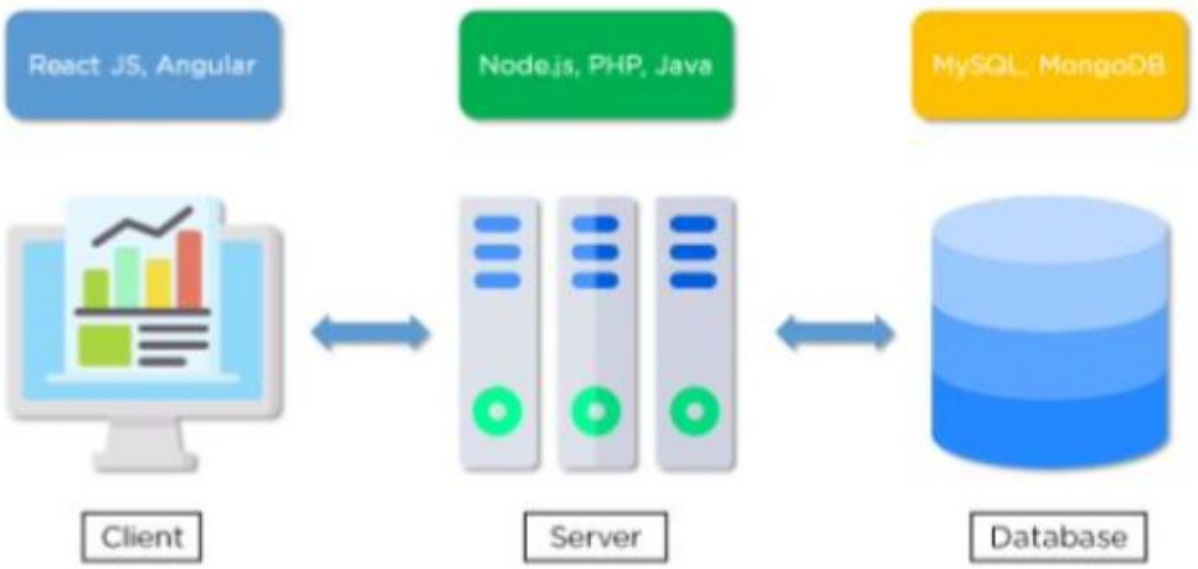


Fig: Web application

Client

The user interacts with the front-end part of a web application. The front-end is usually developed using languages like HTML and CSS styles, along with extensive usage of JavaScript-based frameworks like ReactJS and Angular, which help with application design.

Server

The server is responsible for taking the client requests, performing the required tasks, and sending responses back to the clients. It acts as a middleware between the front-end and stored data to enable operations on the data by a client. Node.js, PHP, and Java are the most popular technologies in use to develop and maintain a web server.

Database

The database stores the data for a web application. The data can be created, updated, and deleted whenever the client requests. MySQL and MongoDB are among the most popular databases used to store data for web applications.

Node.js Server Architecture

Node.js uses the "Single Threaded Event Loop" architecture to handle multiple concurrent clients. Node.js Processing Model is based on the JavaScript event-based model along with the JavaScript callback mechanism.

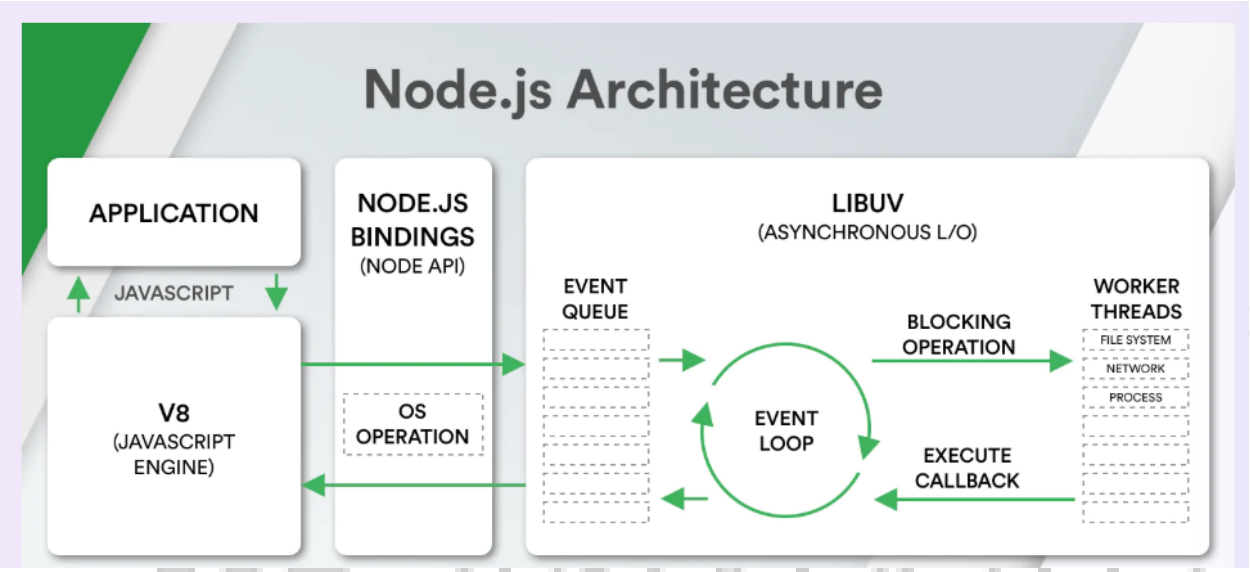


Fig: Node.js architecture

What is Single-threaded event loop?

Node.js runs your JavaScript code on one main thread, which continuously cycles through an event loop checking for tasks, callbacks, and events to process. **Incoming requests are put into an event queue**, and the event loop picks them one by one, executing their associated callback functions on that single thread.

Handling multiple clients

When a request needs slow operations (like file/database access), Node.js delegates those to a background **thread pool (via libuv),** so the main thread does not block and can keep handling other requests.

When these background operations finish, their results are pushed back to the event loop as events, and their callbacks are executed, giving the impression that many tasks run "at the same time" on one thread.

What is Libuv?

libuv **is a C library originally** written for Node.js to abstract **non-blocking I/O operations**.

- Event-driven asynchronous I/O model is integrated.
- It allows the CPU and other resources to be used simultaneously while still performing I/O operations, thereby resulting in efficient use of resources and network.
- It facilitates an event-driven approach wherein I/O and other activities are performed using callback-based notifications.

Parts of the Node.js Architecture:

- Requests

  Incoming requests can be blocking (complex) or non-blocking (simple), depending upon the tasks that a user wants to perform in a web application

  Explanation:

  Blocking (complex) requests

  A blocking request is one where the server has to wait for a slow task to finish before it can move on, such as heavy CPU calculations or synchronous file/database operations.While this task is running, the single Node.js thread is "stuck" and cannot process other requests, which can make the app feel slow or unresponsive.

  Non-blocking (simple) requests

  A non-blocking request uses asynchronous (callback/promise/async-await) APIs for I/O, such as reading files, calling databases, or hitting external APIs.

  In this case, Node.js starts the operation and immediately continues handling other requests; when the result is ready, the callback runs, allowing many clients to be served efficiently.

- Node.js Server

  Node.js server is a server-side platform that takes requests from users, processes those requests, and returns responses to the corresponding users

- Event Queue

  Event Queue in a Node.js server stores incoming client requests and passes those requests one-by-one into the Event Loop

Explanation:

What the event queue does

When a client sends a request, the corresponding callback (for an HTTP request, a timer, or an I/O operation) is added to the event queue instead of being run immediately.

The queue is typically FIFO (first-in, first-out), so callbacks are processed roughly in the order they arrive, which helps keep request handling predictable.

- Thread Pool

  Thread pool consists of all the threads available for carrying out some tasks that might be required to fulfill client requests

  Explanation:

  In Node.js, a thread pool is a group of reusable worker threads that handle heavy or blocking tasks in the background so the main event loop stays free to serve clients. These tasks include things like file system operations, DNS lookups, cryptography, and compression, which might otherwise block the single main thread.

- Event Loop

  Event Loop indefinitely receives requests and processes them, and then returns the responses to corresponding clients.

  Explanation:

  In Node.js, the event loop is a continuously running mechanism that keeps the server alive, always ready to handle new work from clients. It repeatedly checks for pending callbacks and events, executes them, and then goes back to waiting for more, which makes it ideal for long-running servers like HTTP APIs.

- External Resources

  External resources are required to deal with blocking client requests. These resources can be for computation, data storage, etc.

Explanation:

In Node.js, external resources are outside systems or components that handle slow, blocking work so the main event loop can keep serving other clients. These are used whenever a request needs operations that cannot complete instantly, such as I/O or heavy computation.

Difference between asynchronous operation and synchronous operation

Synchronous operation

- In synchronous code, each task runs one after another, and the next line waits until the current operation completes.
- This is called blocking because while a slow task (like reading a big file or querying a database) is running, no other code can progress.

Asynchronous operation

- In asynchronous code, a task is started and the program immediately continues to the next line without waiting for that task to finish.
- This is called non-blocking because slow operations run in the background, and when they complete, a callback, promise, or async/await handler is triggered to process the result.

Node.js Features

Node.js offers a spectrum of features that make it a powerful server-side platform. Some of the key Node.js features are:

1. Event-driven Architecture

Node.js's architecture makes it the perfect platform for developing real-time applications that need to be scalable and highly performant. This design is used by Node.js because it enables programmers to create asynchronous code that can handle a large number of requests and respond to them instantly.

2. Microservices Architecture

A microservices design, in which applications are divided into smaller, easier-to-manage services that can be built, tested, and deployed separately, is a good fit for Node.js. Complex programs can be simpler to grow and maintain using this method.

3. Security

Data is transported safely over the internet thanks to security mechanisms included in Node.js, such as TLS/SSL support. To further improve the security of Node. js-built apps, a large ecosystem of security libraries and tools are available.

4. Non-blocking I/O

The I/O paradigm used by Node.js is non-blocking. It enables the platform to manage several requests concurrently without delaying other ones. With the help of this capability, Node.js apps can handle heavy traffic and data processing without experiencing any lag or crashes.

5. NPM (Node Package Manager)

Developers can quickly manage and install third-party libraries, modules, and frameworks thanks to the robust package management NPM. This can save developers time and effort because they only need to import the necessary package to use it in their code.

6. Large Developer Community

There is a sizable and vibrant developer community for Node.js that produces open-source tools and libraries, offers support, and exchanges expertise. Developers will find it simpler to solve issues and gain knowledge from others as a result.

7. Large Package Ecosystem

With more than 1.5 million packages readily available on the Node Package Manager (NPM), Node.js has a sizable package ecosystem. These packages make it simple for developers to enhance their products and add functionality to their apps.

8. Easy to Learn

Learning Node.js is relatively easy, especially for those who are already comfortable with JavaScript. As a result, developers won't need to learn a brand-new programming language to get started creating apps using Node.js right away.

9. High Scalability

By adding extra nodes to the cluster, Node.js applications can easily grow horizontally, making it simpler to handle increased traffic and data quantities. Because of this, It is perfect for creating highly scalable, high-traffic apps.

10. Cross-Platform Support

It is simple to execute an application on a range of servers and environments thanks to Node.js' cross-platform functionality. Windows, Mac OS, and Linux are just a few of the operating systems that the platform can operate on.

11. Fast Performance

With a built-in V8 engine that converts JavaScript into native machine code, Node.js is intended to be quick. Due to its speed compared to other interpreted languages, it is ideal for creating real-time applications.

Key Differences: Node.js vs. Traditional Web Servers

Feature	Node.js	Traditional Web Servers
Architecture	Single-threaded, non-blocking	Multi-threaded, blocking
Language	JavaScript	Varies (PHP, Python, Ruby, etc.)
Performance	High for I/O-heavy apps	High for CPU-bound apps
Scalability	Excellent with event-driven model	Requires load balancers
Real-time Capabilities	Built-in with WebSockets support	Requires additional setup
Community & Ecosystem	Rapidly growing, npm packages	Mature, but language-dependent

Understanding NPM

NPM (Node Package Manager) is a package manager for NodeJS modules. It helps developers manage project dependencies, scripts, and third-party libraries. By installing NodeJS on your system, NPM is automatically installed and ready to use.

- It is mainly used to manage packages or modules—these are pre-built pieces of code that extend the functionality of your NodeJS application.
- The NPM registry hosts millions of free packages that you can download and use in your project.

- NPM is installed automatically when you install NodeJS, so you don't need to set it up manually.

Package in Node.js

A package in NodeJS is a reusable module of code that adds functionality to your application. It can be anything from a small utility function to a full-featured library.

- Packages can be installed from the NPM registry.
- They are stored in the node_modules folder in your project.
- You can easily install, update, or remove packages with NPM commands

How to Use NPM with NodeJS?

To start using NPM in your project, follow these simple steps

Step 1: Install NodeJS and NPM

First, you need to install NodeJS. NPM is bundled with the NodeJS installation.

Note: *You can follow our article to Install the Node and NPM*

Step 2: Verify the Installation

After installation, verify NodeJS and NPM are installed by running the following commands in your terminal:

```
node -v
npm -v
```

Step 3: Initialize a New NodeJS Project

In the terminal, navigate to your project directory and run:

```
npm init -y
```

This will create a package.json file, which stores metadata about your project, including dependencies and scripts.

Step 4: Install Packages with NPM

To install a package, use the following command

```
npm install <package-name>
```

For example, to install the Express.js framework

```
npm install express
```

This will add express to the node_modules folder and automatically update the package.json file with the installed package information.

What is the Express package in Node.js?

Express (often called **Express.js**) is a **lightweight and flexible web application framework** built on top of **Node.js**. It simplifies the process of building **web servers, REST APIs, and backend applications**.

Step 5: Install Packages Globally

To install packages that you want to use across multiple projects, use the -g flag:

```
npm install -g <package-name>
```

Step 6: Run Scripts

You can also define custom scripts in the package.json file under the "scripts" section. For example:

```
{
   "scripts": {
      "start": "node app.js"
   }
}
```

Then, run the script with

```
npm start
```

Using NPM Package in the project

Create a file named **app.js** in the project directory to use the package

```
//app.js

const http = require('http');//import the required package
const server = http.createServer((req,res)=>{
            res.write("this is server page");
            res.end();
});

server.listen(3000, () => {
```

```
  console.log('Server running at http://localhost:3000');
});
```

Now run the application with

```
node app.js
```

Visit **http://localhost:3000** in your browser,

and you should see the message: **This is server page!**

Updating Packages

Update a Specific Package

npm update express

Update All Packages

npm update

Check Outdated Packages

npm outdated

Uninstalling Packages

npm uninstall express

Removes:

- From node_modules
- From package.json

package.json vs package-lock.json

package.json	package-lock.json
Declares dependencies	Locks exact versions
Editable	Auto-generated
Flexible versions	Ensures consistency

Create a First Web Application in Node.js

Step 1] create folder

Mkdir project-name

Step 2] open project folder

Cd project-name

Step 3] npm init –y

Then it will create package.json

Step 4] install express package

Npm install express

Step 5] create index.js file

```
const express = require("express");
const app = express();
app.get("/", (req, res) => {
        res.write("This is first from node.js program");
        res.end();
});
app.listen(2323, () => {
        console.log("Server has started at http://localhost:2323....");
});
```

Step 6] now run

Node index.js

Now, if you want that quick change affect in output

Then, Repeat from Step 1 to 5

Step 6] install nodemon package

Npm install nodemon –save-dev

And add following code in package.json

```
"scripts": {
 "start": "node index.js",
"dev": "nodemon index.js"
 }
```

Step 7] now run web application

Npm run dev

What is SemVer in NPM?

SemVer, or semantic versioning, serves as a bridge between producers and consumers of packages. It provides important information about software releases and how much risk an upgrade poses. It is called 'Semantic' because each number denotes a different meaning. When

you make changes in an NPM package, SemVer shows the change and its implications. This is useful in change tracking, release management, and version dependencies.

What are the Parts of Semantic Versioning?

Semantic versioning in NPM has three parts: Major, Minor, and Patch. They are essentially numbers separated by dots. These numbers help users understand each version's significance.

Major NPM Version

The major version denotes a backwards-incompatible change to the code. This means that the developer has made breaking changes to the code, hence the name major. The first number denotes the major version in an NPM version code. Therefore, when the major number changes in the version of the node, it implies that the software is incompatible with previous versions.

Let us understand this with an example. Suppose your software is 1.0.0, and after changing certain input parameters, your version is now 2.0.0. This addresses the breaking change you made in the code.

Minor NPM Version

A minor node version refers to adding new features to pre-existing code. These could be improvements to existing features or significant enhancements. They are compatible with older versions, making them backward compatible. The second number in the version code denotes this part.

For example, when the version changes from 1.0.0 to 1.1.0, it indicates a minor change. You can add or subtract any feature and see this change.

Patch NPM Version

The patch version's role is to indicate bug fixes. It is backward compatible and is generally small. When you address minor security issues or fixes in the code, the patch version is updated. The last digit indicates the patch version.

To understand it simply, let us consider a scenario. Suppose you make a minor change by enhancing an already existing software feature. However, you discover there is an issue. To address this bug, you must update your version 1.2.0 to 1.2.1.

What is the Role of the Caret in the NPM Version?

While understanding the three parts of semantic versioning is necessary, knowing what certain symbols mean will help you properly understand the NPM version. There are two main symbols: the Caret (^) and the Tilde (~). A caret (^) symbol before your package.json file indicates

that the software does not allow major version changes. Only minor and patch changes are permissible.

What is the Role of the Tilde in NPM?

The tilde (~) symbol, on the other hand, only allows patch changes. However, all changes should have the same major and minor versions.

Popular Node.js Packages:

1. Express:

Express JS stands out as a simple web framework that simplifies building web applications and APIs in NodeJS. Its unopinionated nature allows users to structure applications according to their preferences, making it a go-to choice for many projects.

To install it in the project run the following command:

```
npm install express
```

Syntax:

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello, Express!');
});

app.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

2. Axios:

Axios simplifies making HTTP requests from both browsers and Node.js applications with its promise-based API. It supports features like request and response interception, automatic JSON parsing, and error handling, making it a preferred choice for handling HTTP communication.

To install it in the project run the following command:

```
npm install axios
```

Syntax:

```
const axios = require('axios');

axios.get('https://jsonplaceholder.typicode.com/posts/1')
  .then(response => {
    console.log(response.data);
  })
  .catch(error => {
    console.error(error);
  });
```

3. Socket.io:

Socket.io facilitates real-time, bidirectional communication between clients and servers, making it ideal for building interactive web applications, chat applications, and multiplayer games.

To install it in the project run the following command:

```
npm install socket.io
```

Syntax:

```
const io = require('socket.io')(http);

io.on('connection', (socket) => {
  console.log('A user connected');

  socket.on('disconnect', () => {
    console.log('User disconnected');
  });
});
```

4. Mongoose:

Mongoose is an elegant MongoDB object modeling tool designed to work in an asynchronous environment like Node.js. It provides a straightforward schema-based solution to model your application data, offering features such as validation, query building, and hooks.

To install it in the project run the following command:

```
npm install mongoose
```

Syntax:

```
const mongoose = require('mongoose');
mongoose.connect('mongodb://localhost:27017/my_database', {
    useNewUrlParser: true, useUnifiedTopology: true
});

const Schema = mongoose.Schema;
const userSchema = new Schema({
  name: String,
  email: String,
});

const User = mongoose.model('User', userSchema);

const newUser = new User({
  name: 'John Doe',
  email: 'john@example.com',
});

newUser.save((err, user) => {
  if (err) return console.error(err);
  console.log('User saved successfully:', user);
});
```

5. Nodemon:

Nodemon is a handy utility that monitors changes in your NodeJS application and automatically restarts the server whenever a change is detected. This eliminates the need to manually stop and restart the server during development, thereby boosting productivity.

To install it in the project run the following command:

```
npm install nodemon --save-dev
```

Syntax:

```
nodemon server.js
```

Unit 2 Introduction to Serverside Framework Node.js

Create HTTP Server in Node JS

Node.js has revolutionized the way we build web applications by allowing developers to create server-side applications using JavaScript. One of its core features is the ability to easily create an HTTP server. In this lesson, we'll dive into the process of creating a basic HTTP server using Node.js.

Importing the HTTP Module

To start our adventure, we need to import the built-in HTTP module. The HTTP module provides essential functionality for creating an HTTP server, including handling incoming requests and sending responses. To import the module, simply use the require() function like so:

```
const http = require('http');
```

Using the http.createServer() Method

Now that we've imported the HTTP module, it's time to use its createServer() method to create an instance of an HTTP server. This method returns an http.Server object, which we can configure to handle incoming requests and send responses.

Here's a basic example of how to create an HTTP server using the createServer() method:

```
const http = require('http');
const server = http.createServer((req, res) => {
 res.writeHead(200, {'Content-Type': 'text/plain'});
res.end('Hello World!');
});
server.listen(3000, () => {
  console.log('Server running at <http://localhost:3000/>');
});
```

In this example, we pass a callback function to the createServer() method. This function will be called each time the server receives an incoming request. The callback function takes two arguments: req (the request object) and res (the response object).

We then call the listen() method on the server object to start listening for incoming connections on port 3000. When the server is ready, the provided callback function is executed, and a message is printed to the console.

Understanding the Request and Response Objects

The req and res objects are instances of the **http.IncomingMessage** and **http.ServerResponse** classes, respectively. They contain valuable information about the incoming request and provide methods for crafting the response.

- The req object (Request): This object contains information about the incoming request, such as the URL, headers, and body. Some of the useful properties of the req object include req.url, req.method, and req.headers.
- The res object (Response): This object represents the server's response to the incoming request. You can use it to set the status code, headers, and response body. Some of the useful methods of the res object include res.writeHead(), res.write(), and res.end().

Let's take a closer look at the example server code from earlier:

```
const server = http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Hello World!');
});
```

In this example, we use the res.writeHead() method to set the response's status code to 200 (indicating success) and the Content-Type header to 'text/plain'. We then use the res.end() method to send the response body ('Hello World!') and signal the end of the response.

By understanding the request and response objects, we can create more complex server logic to handle various types of requests and send appropriate responses.

Binding the Server to a Specific Port and Starting to Listen for Incoming Connections

To make your server accessible, you need to bind it to a specific port and start listening for incoming connections. This is done using the listen() method on the http.Server object. The listen() method takes two arguments: the port number and a callback function to be executed when the server starts listening.

Here's an example:

```
server.listen(3000, () => {
```

```
console.log('Server running at <http://localhost:3000/>');

});
```

In this example, we bind the server to port 3000 and start listening for incoming connections. When the server is ready, the provided callback function is executed, and a message is printed to the console.

Basic Example: Creating a "Hello, World!" HTTP Server

Now that we have covered the fundamental components of creating an HTTP server with Node.js, let's create a simple "Hello, World!" server as a practical example. This server will respond to every incoming request with the text "Hello, World!".

```
// Step 1: Import the HTTP module
const http = require('http');

// Step 2: Create the server using the http.createServer() method
const server = http.createServer((req, res) => {
// Step 3: Set the response headers
  res.writeHead(200, {'Content-Type': 'text/plain'});

  // Step 4: Send the response body
          res.end('Hello, World!');
         });
// Step 5: Bind the server to a specific port and start listening for incoming connections
         server.listen(3000, () => {
           console.log('Server running at <http://localhost:3000/>');
         });
```

In this example, we import the HTTP module, create the server using the createServer() method, set the response headers, send the response body, and bind the server to port 3000.

With this basic example, you now have the foundation to build more complex HTTP servers using Node.js. As you continue to explore Node.js, you can experiment with handling different types of requests, routing, and integrating with databases or other services to create fully-fledged web applications.

The Request Object (req)

- **req.method**: Returns the HTTP method (e.g **GET, POST**, etc.).
- **req.url**: Returns the **requested URL**.
- **req.headers**: Returns an object containing **request headers**.
- **req.on('data')**: Listens for incoming data chunks (useful for **POST requests**).
- **req.on('end')**: Signals the end of the **request** body.

The Response Object (res)

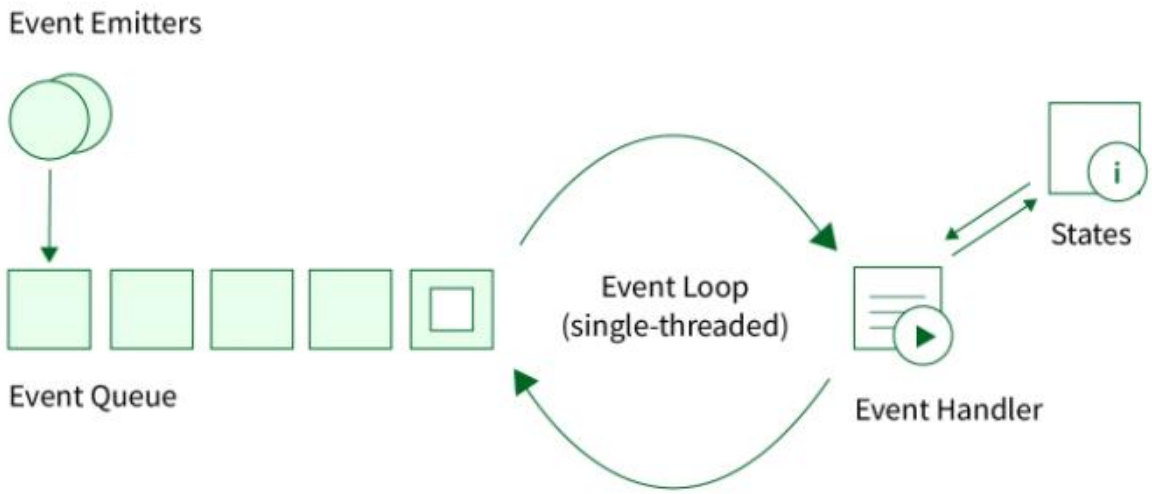
- **res.writeHead(statusCode, headers)**: Sets the **status code** and **response headers**.
- **res.write(data)**: Writes data to the response body.
- **res.end()**: Ends the response.

Common HTTP Status Codes

Code	Message	Description
200	OK	Standard response for successful HTTP requests
201	Created	Request has been fulfilled and new resource created
301	Moved Permanently	Resource has been moved to a new URL
400	Bad Request	Server cannot process the request due to client error
401	Unauthorized	Authentication is required
403	Forbidden	Server refuses to authorize the request
404	Not Found	Requested resource could not be found
500	Internal Server Error	Unexpected condition was encountered

What Are Node.js Events?

Node.js events are notifications that signal the completion of an asynchronous operation or a specific occurrence in the application. These events are emitted by **Event Emitters**, which are objects that can emit events and allow other parts of the application to listen and respond to these events.



Key Concepts:

- **EventEmitter**: This is a core class in Node.js (from the `events` module) that allows objects to emit events and register listeners (callback functions) to handle those events.
- **Emitter**: An object that emits events (e.g., when a file is read or an HTTP request is completed).
- **Listener/Handler**: A function or a set of functions that are invoked when a specific event is emitted.

Common Uses of Events in Node.js:

- **File I/O**: When reading or writing files asynchronously, Node.js will emit events like open, data, end, and error.
- **HTTP Requests**: In a web server, Node.js emits events when a request comes in, or a response is sent, like request and response.
- **Streams**: A stream in Node.js (like reading data from a file) emits events such as data, end, error, and finish.

How to Use Events in Node.js:

To effectively handle events in Node.js, you need to:

Require the Events Module: Node.js has an events module that provides the EventEmitter class.

- const EventEmitter = require('events');

Create an EventEmitter Instance: You can create an instance of EventEmitter to start using events.

- const eventEmitter = new EventEmitter();

Define Event Listeners: Listeners (or handlers) are functions that listen for specific events. You can use the .on() method to register an event listener.

- eventEmitter.on('event', () => { console.log('An event occurred!'); });

Emit Events: You can emit an event using the .emit() method. When the event is triggered, all listeners attached to that event are executed.

- eventEmitter.emit('event'); // This triggers the 'event' listener

For Example

```
// importing events module
const EventEmitter = require('events');

// Creating object of EventEmitter class
var eventEmitter = new EventEmitter();

// Listening to sendMessage event
eventEmitter.on('sendMessage', () => {
  console.log("Hello World!");
});

// Triggering sendMessage event
eventEmitter.emit('sendMessage');
```

Output:

Hello World!

Example with arguments in emit method:

```
// importing events module
const EventEmitter = require('events');

// Creating object of EventEmitter class
var eventEmitter = new EventEmitter();
```

```
// Listening to sendStatus event
eventEmitter.on('sendStatus', (status) => {
  console.log(`Status: ${status}`);
});

// Triggering sendStatus event
eventEmitter.emit('sendStatus',"Complete");
```

Output:

Status: Complete

GET & POST Implementation & Modules

GET and POST are two fundamental HTTP methods: GET is used to request data from a server, while POST is used to send data to the server, usually to create or modify resources. A module is a self-contained, reusable unit of code (or component) that groups related functionality to make programs easier to maintain and scale.

GET method
Purpose: Used to retrieve data from a server without changing its state (e.g., fetching a web page, search results, list of users).

How it sends data: Parameters are appended to the URL as a query string, so data is visible in the address bar (e.g., ?name=John&city=Indore).

Typical use cases: Search forms, filters, read-only API calls like GET /users/42 to fetch a specific user.

POST method
Purpose: Sends data to the server, generally causing a change on the server such as creating or updating data (e.g., submitting a form, creating a post, registering a user).

How it sends data: Data is placed in the request body, not in the URL, which also allows larger and more complex payloads.

Typical use cases: Login forms, registration, file uploads, creating records via APIs like POST.

GET vs POST overview

Aspect	GET	POST
Main use	Retrieve data only.	Send data, create/modify resources.
Where data goes	URL query string, visible in address bar.	Request body, not shown in URL.
Data size	Limited, not ideal for large payloads.	Can handle large payloads and files.

Aspect	GET	POST
Security	Less secure; data visible and logged in URLs.	More secure than GET; body not logged in URL.
Side effects	Should be read-only, no state change.	Typically changes server state.

Connect to NoSQL Database usig Node. JS

1. Overview of NoSQL Databases

NoSQL databases are **non-relational databases** designed to store and process large volumes of data with **high scalability and flexibility**. Unlike traditional relational databases, NoSQL databases do not require fixed schemas and tables.

Key Characteristics:

- Schema-less or flexible schema
- Horizontally scalable
- High availability and fault tolerance
- Optimized for big data and real-time applications

Types of NoSQL Databases:

1. **Document-Oriented** – MongoDB
2. Key-Value Stores – Redis
3. Column-Oriented – Cassandra
4. Graph Databases – Neo4j

Among these, **MongoDB** is most commonly used with **Node.js**.

2. Node.js as a Backend Platform

Node.js is a **JavaScript runtime environment** built on Chrome's V8 engine. It allows JavaScript to run outside the browser and is widely used for backend development.

Important Features of Node.js:

- Asynchronous and non-blocking I/O
- Event-driven architecture
- Single-threaded with high concurrency
- Uses JSON format for data exchange

These features make Node.js highly compatible with NoSQL databases.

3. MongoDB as a NoSQL Database

MongoDB is a **document-based NoSQL database** that stores data in **BSON (Binary JSON)** format.

MongoDB Data Structure:

- **Database** → Collection → Document
- A **collection** is equivalent to a table
- A **document** is equivalent to a row
- Each document has a unique **_id** field

MongoDB does not enforce a strict schema, allowing dynamic fields.

4. Node.js–MongoDB Connection Architecture

To connect Node.js with MongoDB, a **database driver or ODM (Object Data Modeling)** library is used.

Common Options:

1. **MongoDB Native Driver**
   - Low-level access
   - No schema enforcement
2. **Mongoose ODM**
   - Schema-based modeling
   - Built-in validation
   - Middleware and hooks
   - Relationship handling

Mongoose is widely preferred in enterprise applications.

5. Establishing Connection Between Node.js and MongoDB

Connection Steps (Conceptual):

1. Install required packages
2. Import MongoDB/Mongoose module
3. Define connection URI
4. Establish connection
5. Handle connection success or failure

Connection URI Example:

mongodb://localhost:27017/database_name

OR (Cloud Database – MongoDB Atlas):

mongodb+srv://username:password@cluster.mongodb.net/dbname

A successful connection allows Node.js to perform database operations.

6. Schema and Model Concept (Using Mongoose)

Although MongoDB is schema-less, Mongoose introduces **schemas** to ensure data consistency.

Schema:

- Defines structure of a document
- Specifies field types
- Supports validation rules

Model:

- A wrapper around the schema
- Represents a MongoDB collection
- Used to perform CRUD operations

This abstraction simplifies database interaction.

noble UNIVERSITY GLOBAL VISION

CRUD Operations in NoSQL Using Node.js

CRUD operations are fundamental for data manipulation.

Step 1: Install Required Dependencies

Before implementing CRUD operations, the required Node.js packages must be installed.

Common Dependencies:

- **Node.js** – Runtime environment
- **MongoDB / Mongoose** – Database connectivity
- **Express.js** – Web framework (optional but recommended)

Purpose:

These dependencies enable Node.js to communicate with the NoSQL database and handle HTTP requests.

Step 2: Create Node.js Project Structure

A proper project structure improves readability and maintenance.

Typical Structure:

- Server file (app.js / server.js)
- Database configuration file
- Model folder (schemas)
- Routes folder (CRUD APIs)
- Controller folder (business logic)

Purpose:

Separates concerns and follows MVC architecture.

Step 3: Establish Connection with NoSQL Database

- Import MongoDB or Mongoose library
- Define database connection URL
- Establish connection when the server starts
- Handle success and error events

Purpose:

Ensures the Node.js application is connected to the NoSQL database before performing CRUD operations.

Step 4: Define Schema and Model

Although NoSQL databases are schema-less, defining a schema using Mongoose helps maintain data integrity.

Actions:

- Define fields and data types
- Apply validations
- Create a model from the schema

Purpose:

The model acts as a bridge between Node.js and the MongoDB collection.

Step 5: Implement CREATE Operation

Steps:

1. Receive data from client (form or API request)
2. Validate input data
3. Create a new document using the model
4. Save the document in the database
5. Send success or error response

Outcome:

A new record is added to the NoSQL database.

Step 6: Implement READ Operation

Steps:

1. Receive request from client
2. Specify query conditions (if any)
3. Fetch data from the database
4. Format retrieved data
5. Send data as response

Types:

- Read all records
- Read single record by ID
- Conditional read

Outcome:

Stored data is retrieved from the database.

Step 7: Implement UPDATE Operation

Steps:

1. Identify document using unique ID or condition
2. Receive updated data
3. Validate new data
4. Update selected fields or entire document
5. Return update status

Outcome:

Existing record is modified successfully.

Step 8: Implement DELETE Operation

Steps:

1. Identify document to be deleted
2. Confirm deletion condition
3. Remove document from database
4. Send confirmation response

Outcome:

Selected record is permanently removed from the database.